Week 8: Forms and HTTP methods

Reading form data

```
from flask import request

request.args.get("q", "")    # GET: ?q=... in URL
request.form.get("name", "")       # POST: in the body
request.method                     # "GET" or "POST"
request.form.getlist("topping")    # all ticked boxes
```

- Use `.get(key, default)`. `request.form['x']` on a missing field gives **400 Bad Request**.
- Everything from a form is a **string**: convert with `int()` after checking `.isdigit()`.
- A ticked checkbox sends its value; an unticked one sends **nothing**.

GET or POST?

	GET	POST
Purpose	Look (read)	Change (write)
Where	URL query string	Request body
Safe to repeat?	Yes	No: may repeat the change
Bookmark?	Yes	No
Examples	Search, filter, view	Add, edit, delete, login

A **link is always a GET**. Anything that changes data must be a POST form with a button.

One URL, two methods

```
@app.route("/feedback", methods=["GET", "POST"])
def feedback():
    errors, form = {}, {"name": ""}
    if request.method == "POST":
        form = {"name":
            request.form.get("name", "").strip()}
        if not 2 <= len(form["name"]) <= 50:
            errors["name"] = "2 to 50 letters."
        if not errors:
            ...save...
            flash("Thank you!")
            return redirect(url_for("thanks"))
    return render_template("feedback.html",
                           form=form, errors=errors)
```

The recipe

- **GET** → show an empty form.
- **POST** → read the fields, `strip()`, **validate on the server**.
- **Errors?** Show the **same form** again: messages beside the fields, and the values the user typed.
- **No errors?** Do the work, `flash()`, then **redirect**.

Post / Redirect / Get

After a successful POST, `redirect()` to a GET page. Then refresh or Back repeats only a harmless GET, not the POST (no double save).

Flash messages

```
app.secret_key = "dev-only-change-me"   # needed!
flash("Saved.")      # in the route, before redirect

{% for m in get_flashed_messages() %}
  <p class="flash">{{ m }}</p>
{% endfor %}          {# put in base.html #}
```

Form snippets in Jinja

```
<input type="text" name="name"
       value="{{ form.name }}">
{% if errors.name %}
  <p class="error">{{ errors.name }}</p>{% endif %}

<option value="{{ n }}"
  {{ 'selected' if form.rating == n|string }}>
  {{ n }}</option>
<input type="radio" name="g" value="F"
  {{ 'checked' if form.g == 'F' }}>
```

Never trust the form. A `<select>` only offers some choices, but anyone can send anything: check the value on the server. `required` and `type="email"` are conveniences, not safety.

Errors you will meet

Error	Usual cause
400 Bad Request	`request.form['x']` for a missing field
405 Method Not Allowed	Route lacks `methods=["POST"]` (or GET sent to a POST route)
`RuntimeError ... secret key`	`flash` used without `app.secret_key`
Page saves twice on refresh	No redirect after POST